Agentic AI Interview Preparation

100 Questions & Interview-Ready Answers

A practical preparation guide covering AWS Bedrock, LLMs, AI agents, multi-agent systems, RAG, embeddings, vector databases, React, tools, security, prompt injection, RBAC, production engineering and leadership.

Designed for Senior Engineering Manager and architecture interviews

Contents

Section 1: AI, LLM and Agentic AI fundamentals (1–15)

Section 2: AI agents, tools and orchestration (16–30)

Section 3: RAG, embeddings and vector databases (31–45)

Section 4: Multi-agent systems (46–60)

Section 5: Security, Guardrails, RBAC and prompt injection (61–75)

Section 6: React UI and end-to-end AWS architecture (76–85)

Section 7: Advanced production engineering and leadership (86–100)

Suggested answer pattern: explain the concept, give a practical example, describe the architecture, and close with security, reliability, cost or trade-offs where relevant.

Section 1: AI, LLM and Agentic AI fundamentals

1. What is Generative AI, and how is it different from traditional AI and machine learning?

Traditional AI and machine learning are commonly used to predict outcomes, classify data, detect anomalies or make recommendations. Generative AI produces new content, such as text, code, images and summaries, based on patterns learned during training.

For example, a traditional ML model might predict whether a loan applicant is high risk. A generative AI system could summarize the applicant's documents and explain the factors relevant to the assessment.

In an enterprise application, I would use traditional ML for well-defined prediction tasks and Generative AI for language-heavy tasks such as summarization, document understanding and natural-language interaction. These approaches can also be combined.

2. What is an LLM, and how does it generate responses?

A Large Language Model is a neural network trained on large amounts of data to predict and generate token sequences. Given an input prompt, it processes the context and generates a response token by token.

For example, if I send a contract to Claude through AWS Bedrock and ask it to summarize the termination clauses, the model interprets the document and generates a natural-language summary.

An LLM does not automatically know our latest internal business data. For that, we may need RAG, tools, or integration with enterprise systems.

3. What are tokens, tokenization, context windows and temperature?

Tokens are units of text processed by a model; a token may represent a word, part of a word or punctuation. Tokenization converts text into tokens. A context window is the maximum input and output context a model can handle in a request, subject to that model's limits. Temperature is a sampling parameter that influences variability: lower values generally produce more predictable output, while higher values generally allow more variation.

For example, an agent processing a 300-page policy manual should not blindly send the entire document on every request. RAG can retrieve relevant sections, reducing unnecessary context and often lowering cost and latency.

4. What is the difference between Generative AI, an AI assistant, an AI agent and agentic AI?

Generative AI is the underlying capability to generate content. An AI assistant provides an interface for helping users. An AI agent can select and execute tools to accomplish a task. Agentic AI describes systems that use planning, decisions and actions across one or more steps to pursue a goal.

Consider an IT incident: a generative AI model summarizes the incident; an assistant answers questions; an agent retrieves monitoring data and queries recent deployments; an agentic workflow investigates multiple signals, recommends remediation and, with appropriate authorization, executes an approved action.

The key distinction is not simply whether the system uses an LLM, but how much task execution and decision-making it performs.

5. What are the essential components of an AI agent?

A typical agent includes:
- LLM: interprets the request and selects an action.
- Instructions: define its role, objectives and constraints.
- Tools: APIs, Lambda functions or other capabilities it can invoke.
- State and memory: preserve relevant task information and conversation context.
- Orchestration: controls execution, tool results, retries and stopping conditions.
- Security controls: enforce permissions, validate actions and protect data.
- Observability: records execution traces, errors, latency and cost.

For an HR assistant built with React and AWS Bedrock, the LLM interprets the employee's question, a retrieval tool searches authorized HR policies, and the application validates access before returning an answer. Memory is optional; not every agent needs persistent memory.

6. How does an agent decide whether to answer directly or call a tool?

The model receives the user's request, its instructions, available tool descriptions and relevant context. It can generate a direct response or request a tool call.

For example, “What is our annual leave policy?” may trigger retrieval of an HR policy document. “How many leave days remain in my account?” requires an authorized HR API call.

The application—not the model itself—executes the requested function, validates the result and returns it to the model if further processing is needed. Authorization and business rules must be enforced independently; do not rely solely on the model to decide whether a sensitive action is allowed.

7. What is the difference between deterministic automation, an LLM workflow and an autonomous agent?

Deterministic automation follows predefined rules and steps. An LLM workflow uses an LLM within a predefined process, such as extracting invoice fields and validating them. An AI agent selects tools or actions dynamically based on the task and available information.

For invoice processing, deterministic code can validate totals, an LLM can extract data from an unstructured invoice, and an agent can investigate a discrepancy by retrieving the purchase order and checking receipt records.

Keep calculations, payment controls and critical business rules deterministic wherever possible. Agentic behavior is most useful when the next step depends on information discovered during execution.

8. What is the ReAct pattern?

ReAct stands for Reasoning and Acting. It is a pattern in which an LLM alternates between deciding what to do, invoking an action and using the returned observation to determine the next step.

For an IT outage, an agent might call a monitoring API, inspect an error spike, query recent deployments, correlate the results and recommend a rollback.

The orchestration layer manages the action-observation loop and enforces iteration limits. The application can record concise decision summaries and tool traces without exposing private internal reasoning.

9. What is the difference between reasoning, planning and execution?

Reasoning interprets the problem and assesses available information. Planning determines the sequence of steps needed to achieve a goal. Execution calls tools and carries out those steps.

For a sales account review, an agent may identify missing information, plan to retrieve CRM records and recent account activity, then execute those API calls before generating a report.

For enterprise systems, explicit task plans and observable execution states are easier to test, audit and recover than an unrestricted loop.

10. What is the difference between prompt engineering, fine-tuning, RAG and tool calling?

Prompt engineering guides model behavior; fine-tuning adapts model behavior using training examples; RAG supplies relevant external information at inference time; tool calling accesses systems or performs actions.

Examples: use a prompt for structured JSON output, fine-tuning for a specialized response style if evaluation justifies it, RAG for the latest HR policy, and a tool to retrieve a live leave balance.

These approaches are complementary. For an enterprise policy assistant, begin with prompts and RAG, add tools for live data, and consider fine-tuning only when evaluations show a persistent need that prompts and retrieval do not address.

11. What is AWS Bedrock, and why use it?

Amazon Bedrock is a managed AWS service for building generative AI applications using supported foundation models and related capabilities. It provides model invocation APIs and features such as Knowledge Bases, Guardrails and agent capabilities, depending on model and configuration.

I would consider Bedrock when an organization wants to integrate generative AI with its AWS environment, apply centralized security controls and avoid managing foundation-model infrastructure directly. I would still evaluate model quality, availability, regional requirements, cost and service-specific limitations.

12. How do you choose a foundation model on Bedrock?

Compare models using a representative evaluation dataset. Criteria include task accuracy, instruction following, tool-calling reliability, structured-output quality, context-window requirements, latency, throughput, token cost, regional availability and organizational requirements.

A simple document classifier may work with a smaller, lower-cost model, while a complex incident investigation may justify a more capable model. Benchmark models on realistic requests, including ambiguous inputs and tool failures, and choose the least expensive model that meets quality and reliability thresholds.

13. What is the difference between direct model invocation, Bedrock Agents and a custom agent?

Direct model invocation gives the application control over prompts and the request-response cycle; it suits summarization, extraction and bounded tasks. Bedrock Agents provide managed agent capabilities for coordinating model-driven tasks with configured action groups and knowledge sources. A custom agent uses application-controlled orchestration—potentially Lambda, Step Functions or a framework—to manage planning, tools, retries and business rules.

A policy summarizer may need direct invocation; a straightforward assistant may fit Bedrock Agents; a regulated multi-agent workflow with complex approvals and recovery may justify custom orchestration. Decide based on control, complexity, maintainability and operational needs.

14. How do you manage the LLM context window?

Do not treat the context window as a substitute for information management. Retrieve relevant chunks through RAG, summarize older conversation history, keep structured task state separate from conversation history, remove duplicate tool results, limit retrieved chunks and tailor context to each task stage.

An incident agent can retain a compact summary, discovered facts and completed steps instead of resending every tool response on every iteration. Preserve essential evidence and authorization information rather than summarizing away critical details.

15. Design an HR assistant that answers policy questions and escalates unresolved questions.

Use a React interface backed by an authenticated API and an AWS-based RAG service. Authenticate the employee; retrieve only policy content they may access; use Bedrock to generate a grounded answer; apply configured safety controls; return the answer with source citations; and escalate when evidence is insufficient or human judgment is required.

The architecture includes React, API Gateway/Lambda, identity and authorization, Bedrock, a retrieval layer and an authorized HR escalation tool. Evaluate retrieval quality, groundedness, access-control failures, escalation accuracy and latency. Never allow the LLM to invent policy details or decide independently whether the employee can access restricted information.

Section 2: AI agents, tools and orchestration

16. What is a tool in an AI agent?

A tool is a capability an agent can invoke to interact with an external system or perform an operation, such as querying a database, calling a REST API, searching documents or updating a record.

The application sends the request and tool definitions to the LLM; the LLM returns a tool-call request; the application validates arguments and authorization; the application executes the tool; and the result is returned to the LLM. The LLM selects the tool, but the application executes it. This distinction is fundamental to agent security.

17. How do you expose a business capability through AWS Lambda?

Create a Lambda function behind an API or configure it as a Bedrock Agent action-group executor. For example, a getInvoiceStatus tool accepts an invoice ID and returns its payment status.

Use a clear input schema, server-side validation, authentication and authorization, least-privilege IAM, timeouts, controlled errors and structured responses. Keep business logic in a service layer rather than embedding critical rules in prompts. Reuse the validated service from the agent and other applications.

18. What is the difference between an API, a tool and a Bedrock Agent action group?

An API exposes a software capability through a defined interface. A tool is the capability presented to an agent, with a description and input/output contract. A Bedrock Agent action group groups related actions and defines how the agent invokes them, commonly through an API schema and Lambda executor.

A customer-management API may expose customer lookup and contact updates; the agent can use these through a configured action group. The API defines what the system can do, while the tool definition helps the model understand when and how to request it.

19. How does function calling work with an LLM?

Function calling lets a model produce a structured request to invoke a function rather than just natural-language text. For example, it may request getInvoiceStatus with an invoice ID.

The application validates the request, invokes the actual function and passes the result back to the model. A function-call request does not prove the operation succeeded. The application must execute it and verify the outcome, especially for payments, refunds and customer-record updates.

20. Design an agent with customer lookup, invoice retrieval and payment-status update tools.

Expose lookupCustomer(customerId), getInvoice(invoiceId) and updatePaymentStatus(invoiceId, status). If a user asks why an invoice has not been paid, retrieve the invoice and relevant customer information before explaining the status.

If the user asks to update the status, verify permissions and business rules. Do not infer that a payment occurred merely because the user requested an update. For sensitive changes, require approval and verify the final state through an authoritative business API.

21. How do you handle tool failures and timeouts?

Distinguish transient failures from permanent failures. Retry transient errors with bounded exponential backoff and jitter. For timeouts, determine whether the operation may already have completed. Correct safe validation errors or ask for clarification. Stop on authorization failures. Explain permanent business errors and offer an appropriate next step.

If a payment API times out, check payment status using an idempotency key or transaction reference before submitting another payment. Give the agent structured errors, not internal stack traces or secrets.

22. How do you prevent infinite tool-calling loops?

Enforce maximum tool calls and iterations, an overall deadline, per-tool timeouts, repeated-call detection where appropriate, token and monetary budgets, and explicit completion/escalation conditions.

If an incident agent repeatedly queries the same monitoring endpoint without new information, the orchestrator should stop and return the evidence gathered so far. The application—not just the model—must enforce hard limits.

23. How do you maintain state across agent calls?

Separate conversation history, task state and persistent business data. Conversation history holds relevant user messages and responses. Task state holds the objective, completed steps, pending actions and tool results. Persistent business data belongs in an authoritative database or system.

An incident task might store its incident ID, findings, completed investigations and pending approval in a durable task record. Do not rely on Lambda's in-memory state for long-running tasks. Minimize, encrypt and access-control sensitive information.

24. How do you implement human approval before a sensitive action?

Separate proposing an action from executing it. The agent prepares a structured proposal with the operation, target, evidence and impact. The backend creates a pending approval request and returns an approval ID to React.

An authorized reviewer approves or rejects it. Before execution, validate reviewer permissions, approval validity and whether the business state changed. Bind approval to the specific action and parameters, expire stale approvals and audit the outcome. An LLM statement that approval was received is not authorization.

25. What is the difference between an agent and AWS Step Functions?

An agent uses model-driven decisions to interpret requests, select tools and adapt to observations. Step Functions orchestrates defined workflows with explicit states, branching, retries, error handling and execution history.

An agent may investigate an invoice discrepancy and propose a resolution; Step Functions can manage deterministic approval, record update and notification stages. Combine them when flexible interpretation and predictable execution are both needed.

26. How do you distinguish read-only tools from write or destructive tools?

Define separate permissions and execution paths. Read tools retrieve authorized information; write tools modify records; destructive or high-impact tools delete records, transfer money or trigger consequential operations.

Write tools need stronger validation, authorization and potentially approval. A getCustomerDetails tool should not share the same authorization boundary as deleteCustomer. IAM, service-side authorization and business rules must enforce the separation regardless of the LLM's request.

27. How do you implement idempotency for agent tool calls?

Idempotency ensures retrying the same logical operation does not cause unintended duplicate effects. For payments, persist an idempotency key with the operation status. A repeated request returns the existing result instead of creating a second payment.

Use transaction identifiers, database constraints and reconciliation with the authoritative system. Because an agent may retry after a timeout without knowing whether the first attempt succeeded, idempotency must be implemented by the service handling the operation, not merely described in a prompt.

28. How do you handle long-running agent tasks and Lambda execution limits?

Separate task initiation from execution. The API accepts a request, creates a task ID and starts asynchronous processing. Lambda functions perform bounded work while Step Functions or another durable orchestrator manages the workflow.

React receives the task ID, polls status or receives updates, and retrieves the final result. Persist progress so browser closure or reconnection does not lose task state. This avoids relying on one Lambda invocation to run the entire task.

29. How do you design effective tool descriptions and schemas?

Descriptions should specify the tool's purpose, required arguments, expected result and limitations. For example, getInvoiceStatus retrieves an existing status and must not modify the invoice.

Use strict schemas, required fields, enumerated values where useful and server-side validation. Ask for clarification when identifiers are ambiguous. Test tool selection against similar names, incomplete arguments and adversarial requests. Good descriptions improve selection, but authorization and validation remain independent controls.

30. Design an IT incident-resolution agent.

Use a React incident console, API and orchestrator, a Bedrock investigation agent, monitoring and deployment APIs, a RAG knowledge base, and human approval for high-impact remediation.

The agent retrieves metrics, checks recent deployments, searches runbooks and correlates results. It returns evidence, likely causes, uncertainty and recommendations. Read-only investigation can be automatic. Restarts or rollbacks require explicit policies, least-privilege access and appropriate approval.

Evaluate diagnostic accuracy, time to recommendation, safe tool use, false remediation proposals and successful incident resolution.

Section 3: RAG, embeddings and vector databases

31. What is RAG, and why is it needed?

Retrieval-Augmented Generation combines information retrieval with an LLM. The application retrieves relevant information from an external knowledge source and includes it in the model's context instead of relying only on pretrained knowledge.

An HR assistant can retrieve the latest leave policy before answering. RAG provides access to current enterprise information and source-based answers without retraining, but it does not eliminate hallucinations. Retrieval quality, document quality and grounding must be evaluated.

32. Explain the complete RAG pipeline.

Ingestion: collect documents (for example, from S3); extract text and structure, using OCR if needed; split documents into meaningful chunks; generate embeddings; and store vectors, text and metadata.

Query: authenticate the user; embed the question; retrieve relevant chunks; apply metadata filters and optionally rerank; provide selected context to the LLM; and generate a grounded answer with citations.

Bedrock Knowledge Bases can provide managed ingestion and retrieval, while a custom pipeline offers more control over chunking, access enforcement and ranking.

33. What are embeddings?

Embeddings are numerical vectors representing information such as text. An embedding model maps text into a vector space where semantic relationships can be measured. “How much annual leave do I have?” and “What is my remaining vacation allowance?” may have similar embeddings.

During ingestion, generate embeddings for document chunks. At query time, embed the question and retrieve relevant chunks. Embeddings do not guarantee identical meaning; model, language, domain and retrieval strategy affect quality.

34. What is the difference between an embedding model and a generative LLM?

An embedding model transforms text into numerical vectors for semantic search, clustering and similarity comparison. A generative LLM generates or transforms content, such as answers, summaries or tool calls.

In RAG, the embedding model helps retrieve information while the generative model uses that information to formulate an answer. They may come from the same vendor but serve different purposes and must be evaluated independently.

35. What is a vector database? Compare Amazon OpenSearch and Bedrock Knowledge Bases.

A vector database stores embeddings and supports similarity retrieval, often with metadata filtering and keyword or hybrid search. Amazon OpenSearch can serve as a vector-search engine with control over indexing and retrieval.

Amazon Bedrock Knowledge Bases is a higher-level managed RAG capability that coordinates supported data sources, parsing/chunking options, embedding generation and retrieval, using supported backends. Knowledge Bases is a managed workflow; OpenSearch is a search/storage technology that can underpin custom RAG.

Choose based on operational overhead, retrieval needs, scale, access controls and customization requirements.

36. Explain cosine similarity, dot product and Euclidean distance.

Cosine similarity measures the angle between vectors; dot product combines magnitude and directional alignment; Euclidean distance measures straight-line distance. The appropriate choice depends on the embedding model and how its vectors are intended to be compared. Normalization affects the relationships between these metrics.

Use the metric recommended for the selected embedding model and validate retrieval against representative data rather than assuming one metric is universally best.

37. How do you choose chunk size and overlap?

Chunk size should preserve enough context to answer a question without retrieving large amounts of irrelevant information. Small chunks improve precision but can lose context; large chunks preserve context but may dilute relevance and consume more tokens.

For HR policies, split by headings and clauses rather than arbitrary character boundaries. For contracts, keep clauses and relevant definitions together. Benchmark chunking strategies using retrieval precision/recall, groundedness, latency and cost. Overlap can preserve continuity, but excessive overlap duplicates context.

38. What is the difference between semantic search, keyword search and hybrid search?

Keyword search matches terms, semantic search uses embeddings to find meaning-based similarity, and hybrid search combines both. Exact invoice identifiers benefit from lexical matching, while questions such as “Why is my supplier payment delayed?” benefit from semantic retrieval.

Hybrid retrieval is useful for knowledge bases containing prose, identifiers, product names and structured references. Evaluate ranking quality rather than assuming hybrid search is always better.

39. How do metadata filtering and access-control filtering work in RAG?

Metadata filtering restricts retrieval by attributes such as department, document type, region or tenant. Access-control filtering ensures users retrieve only documents they are permitted to see.

An HR assistant may expose general leave policies to all employees but restrict compensation documents to authorized HR personnel. The backend must derive identity and permissions from trusted authentication, enforce filters server-side and avoid trusting client-supplied tenant IDs. Test negative cases to ensure unauthorized content cannot appear in results, citations or answers.

40. What is reranking in RAG?

Reranking reorders candidate documents according to relevance to the query. Initial search might retrieve 50 chunks; a reranker evaluates them and selects the most relevant five or ten for the LLM.

It can improve precision when initial retrieval returns semantically similar but not sufficiently useful content. The trade-off is additional latency and cost, so measure quality improvements against expense.

41. How do you process PDFs, tables, scanned documents and images?

Use text parsing for text PDFs, OCR for scans, structure-aware extraction for tables, and image understanding or multimodal processing for images. Complex financial or contractual documents may require layout-aware extraction and validation.

Flattening an invoice table into plain text can destroy row/column relationships. Validate extracted content, retain document and page references, and evaluate extraction accuracy before embedding it.

42. How do you keep a RAG knowledge base up to date?

Use ingestion pipelines to detect new, modified and deleted documents. Reprocess changed content, regenerate embeddings when required, update the index and remove or disable deleted chunks. Track versions, ingestion status, failures and processing timestamps.

For sensitive data, access revocation must not wait for eventual reindexing. Use an authorization or revocation mechanism to prevent stale entries from remaining accessible.

43. Why can RAG still hallucinate?

RAG can retrieve irrelevant, incomplete, outdated or contradictory documents. Even with relevant context, the LLM may misinterpret evidence or generate unsupported details. Retrieving a policy for the wrong country, for example, can produce an incorrect leave entitlement.

Improve retrieval evaluation, metadata filtering, version control, grounding instructions and output checks. If evidence is insufficient, the system should say so instead of fabricating an answer.

44. How do you evaluate RAG quality?

Evaluate retrieval and generation. Retrieval measures include precision/recall at k and ranking quality such as NDCG. Generation measures include relevance, faithfulness, correctness, completeness, citation accuracy and groundedness.

Build a representative dataset with known supporting documents, including ambiguous and unanswerable questions. Automated evaluation speeds regression testing, but use human review for high-impact domains and validate automated judges against human assessments.

45. Design a production RAG system for 100,000 documents with strict permissions.

Use S3 or enterprise repositories as sources, an ingestion pipeline for extraction and chunking, an embedding model and a managed or custom vector backend. At query time, authenticate the user, resolve access scope and perform filtered retrieval. Pass only authorized evidence to the LLM and return source references.

Include incremental ingestion, versioning, document-level access controls, encryption, least-privilege IAM, ingestion monitoring, evaluation datasets, deletion/revocation workflows, scaling, backup and recovery. Document count alone does not determine architecture; document size, update frequency, query volume, concurrency, security and quality requirements matter.

Section 4: Multi-agent systems

46. What is a multi-agent system, and when is it better than a single agent?

A multi-agent system uses agents with different responsibilities to complete a larger task. Each may have specialized instructions, tools or data access. A sales-intelligence system might use research, CRM-analysis and report-generation agents.

Use multiple agents when separation and specialization measurably improve quality, security or maintainability. If one agent with several tools solves the task reliably, multi-agent architecture may add unnecessary latency, cost and failure points.

47. What is a supervisor-agent architecture?

A supervisor coordinates specialists: it interprets the request, delegates tasks, collects results and decides next steps. A sales-account supervisor might delegate company research and CRM analysis, then combine findings into a report.

Keep the supervisor responsible for coordination and specialists narrowly scoped with explicit tool permissions. The orchestration layer should validate results, enforce execution limits and handle failures.

48. Centralized versus decentralized multi-agent architecture?

A centralized architecture uses a supervisor to coordinate tasks, improving control, auditability and failure handling. A decentralized architecture lets agents coordinate more independently, but increases complexity around ownership, consistency, security and termination.

For a regulated enterprise workflow, start with centralized orchestration because it is easier to observe and govern. Consider decentralized coordination only when there is a demonstrated benefit.

49. Design research, data-analysis and report-generation agents.

The research agent retrieves evidence; the data-analysis agent processes authorized structured data and produces validated metrics; the report-generation agent combines evidence and metrics. Research and analysis can run in parallel if independent, while reporting waits for both.

Pass structured outputs containing facts, source references, timestamps and errors rather than only free-form text. This improves validation and makes discrepancies easier to investigate.

50. How do agents communicate?

Agents can communicate through orchestrator-mediated requests, structured messages, shared task state or asynchronous queues. A research agent might return JSON with facts and citations for a report agent.

Prefer explicit message contracts and orchestrator-controlled handoffs over unrestricted conversations. Shared state needs clear ownership and access policies to avoid conflicting updates or information leakage.

51. When would you use sequential, parallel or hierarchical execution?

Sequential execution is appropriate when each step depends on the previous result. Parallel execution fits independent tasks. Hierarchical execution is useful when a supervisor decomposes a complex task into subtasks that may need their own orchestration.

Company research and internal CRM retrieval can run in parallel; final report generation waits for both. Parallelism reduces latency but increases concurrent cost and complicates error handling, so impose concurrency limits and tolerate partial failures.

52. How should a supervisor select a specialist agent?

Use deterministic routing for clear, stable categories and LLM-based routing for ambiguous requests needing semantic interpretation. A hybrid router might classify HR versus finance questions, then apply deterministic identity, permission and capability rules.

Evaluate routing accuracy, false delegation, latency and failures. A supervisor must not delegate merely because an agent appears capable; it must also verify authorization.

53. How do you prevent agents from conflicting over the same business record?

Enforce consistency at the data or service layer using optimistic locking, transactions, version checks, idempotency keys or distributed locks, depending on the operation.

If two agents update a customer record, an API can require an expected record version and reject stale writes. Agent coordination helps, but the authoritative system must enforce consistency.

54. How do you manage shared memory securely across agents?

Separate shared task context from persistent business data and restrict both by purpose and permission. The supervisor might share an incident ID and findings with an authorized specialist without sharing unrelated customer information.

Use task-scoped state, explicit data contracts, tenant isolation and access checks. Avoid credentials and unnecessary sensitive data in messages. Shared memory must not let one agent bypass another system's access controls.

55. What happens when a specialist agent fails or returns an incorrect result?

Validate outputs against expected schemas and evidence requirements. Retry transient infrastructure failures within limits; request correction or use a fallback for invalid responses; seek independent evidence or escalate for important factual disagreements.

If a finance agent's invoice amount conflicts with the source system, query the authoritative invoice API instead of asking another LLM to vote. Track partial results so recovery does not repeat every completed task.

56. What are the challenges of multi-agent systems?

Challenges include higher token cost and latency, complex debugging, inconsistent outputs, cross-agent permission management and cascading failures. Control them through narrow responsibilities, bounded delegation, structured contracts, deterministic orchestration and end-to-end tracing.

Adopt multi-agent architecture only when evaluation demonstrates measurable value over a simpler solution.

57. How would you implement a multi-agent system with AWS Bedrock and Lambda?

React submits a request to API Gateway; Lambda authenticates it and creates a task; the supervisor delegates to specialists; specialists invoke authorized tools through Lambda or APIs; results are validated and aggregated; and the final response and execution details are returned and recorded.

Use synchronous orchestration for short tasks and consider Step Functions plus durable state for long-running workflows. Managed Bedrock capabilities can reduce custom orchestration, but verify supported collaboration patterns and constraints.

58. Bedrock multi-agent collaboration versus a custom orchestrator?

Managed Bedrock agent capabilities simplify supported delegation patterns. A custom orchestrator offers greater control over routing, state transitions, retries, approvals, concurrency, tool validation and audit requirements.

A research-and-summary assistant may fit managed orchestration. A payment workflow with transaction-level approvals and strict state transitions may justify custom orchestration. Compare control, operational effort, integration complexity, maintainability and platform constraints.

59. Design a multi-agent sales-intelligence system.

A supervisor coordinates a research agent, a CRM-analysis agent and a reporting agent. Research and CRM agents work in parallel; the supervisor validates findings before reporting.

Use RAG for internal sales playbooks, APIs for current CRM data and React for the interface. Enforce permissions for both documents and CRM records. Measure report accuracy, citation quality, task time, tool success and sales-team adoption.

60. How do you resolve disagreement between specialist agents?

Do not simply take a majority vote; agents may share the same faulty source or assumptions. Compare evidence, timestamps, source reliability and definitions. Retrieve authoritative data or apply deterministic validation when necessary.

If two agents report different revenue figures, check reporting period, currency, source and whether one represents a subsidiary. If disagreement remains unresolved, communicate uncertainty rather than inventing consensus.

Section 5: Security, Guardrails, RBAC and prompt injection

61. What are AWS Bedrock Guardrails?

Bedrock Guardrails provides configurable controls to help filter or assess model inputs and outputs. Depending on configuration, features can include content filters, denied topics, sensitive-information handling, grounding checks and prompt-attack detection.

An HR assistant can apply policies to reduce harmful content and detect certain prompt attacks. Guardrails are one layer, not a replacement for authentication, authorization, validation or secure tool execution. Test realistic attacks and monitor false positives and missed detections.

62. Input guardrails versus output guardrails versus application security?

Input guardrails assess incoming prompts; output guardrails assess generated content; application security enforces identity, permissions, data access and business rules.

An input control may detect an attempt to override instructions, backend authorization prevents access to another employee's salary, and output controls reduce sensitive disclosure. All are necessary: content filtering cannot substitute for authorization, and authorization alone does not ensure safe or accurate output.

63. What is prompt injection?

Prompt injection is an attack where untrusted input attempts to manipulate an LLM or override intended instructions. Direct injection comes from a user; indirect injection comes from content the model processes, such as a malicious PDF, webpage or tool response.

Treat retrieved documents and tool outputs as untrusted data, keep trusted instructions separate, restrict tools and require server-side authorization for sensitive operations.

64. How do you defend RAG against prompt injection hidden in a PDF?

Treat document text as untrusted evidence, not trusted instructions; separate application instructions from retrieved content; apply relevant detection controls; limit available tools; enforce authorization independently; require approval for sensitive actions; and test malicious documents.

If a PDF says to email all employee records externally, the agent should treat it as content and lack unrestricted ability to perform the action. No prompt or guardrail guarantees complete prevention; ensure injection cannot grant permissions or bypass trusted services.

65. Can Bedrock Guardrails completely prevent prompt injection?

No. Guardrails can detect or filter certain attack patterns but cannot guarantee detection of every prompt injection, including disguised or indirect attacks.

Combine Guardrails with tool allowlists, least-privilege IAM, server-side authorization, retrieval filtering, output validation and human approval for high-impact actions. Test end-to-end exploit scenarios because text may look safe while the agent still attempts an unsafe tool call.

66. Explain authentication, authorization and RBAC.

Authentication establishes who the user is; authorization determines what the identity may do; RBAC assigns permissions through roles such as employee, HR administrator or finance approver.

An employee may access their own leave balance while an HR administrator has broader access. In a React/Bedrock application, the identity provider authenticates, the backend establishes trusted identity and authorization is enforced before every sensitive operation. The model must never be the final authority on permissions.

67. How do you propagate identity from React to downstream systems?

Authenticate through Cognito or an enterprise identity provider. React sends an access token to API Gateway, where a configured authorizer validates it. The backend obtains trusted identity and determines permissions for retrieval and tool execution; downstream services enforce their own access requirements or receive appropriately scoped identity context.

Do not trust a user ID in the request body. A user must not change employeeId in the payload to retrieve another employee's leave balance. Keep service credentials and user identity separate.

68. Why is hiding a button in React not sufficient authorization?

React is controlled by the user; they can alter the interface or call APIs directly. Hiding an administrative button does not secure the API.

Enforce permissions at API Gateway where appropriate, in backend authorization and in the downstream service. A manually submitted delete request must be rejected unless the authenticated identity has the required permission. Client checks improve usability; server checks provide the security boundary.

69. How do you implement document-level RBAC in a vector database?

Associate each document/chunk with metadata such as tenant, department, document ID and classification. The backend derives the user's permitted scope and applies filters before retrieved content reaches the LLM.

For sensitive systems, verify that filters are enforced by a trusted retrieval layer and cannot be removed by user input or model-generated parameters. Test whether unauthorized chunks can appear in intermediate results, citations, caches or logs. Metadata filtering alone may not provide complete isolation.

70. What is the confused-deputy problem in AI agents?

A confused deputy is a privileged component tricked into using its authority for someone not entitled to the action. For example, a Lambda with permission to read all employee records may be manipulated by a user to retrieve another employee's salary.

Mitigate with user-context authorization, narrow service permissions, resource-level checks, allowlisted operations and approval for sensitive actions. A Lambda's permission to perform an operation does not mean every user may request it.

71. How do you prevent data leakage between tenants?

Enforce tenant isolation throughout authentication, conversation storage, RAG retrieval, tool execution, caching and logs. Derive the tenant from trusted identity rather than user or LLM input. Depending on risk, use tenant-scoped indexes, separate resources or rigorously enforced filters. Cache keys must include relevant tenant and authorization context.

Test cross-tenant access and ensure documents, citations, tool results and histories cannot cross boundaries.

72. How do you protect PII and other sensitive data?

Minimize sensitive data sent to the LLM; use redaction or masking where appropriate; encrypt in transit and at rest; manage secrets securely; apply least privilege; configure retention and logging carefully; and validate outputs.

An invoice agent may need an invoice ID and amount but not supplier bank credentials. Guardrails can help with sensitive-information filtering, but do not replace control over what is retrieved, transmitted or logged.

73. How do you prevent unauthorized commands or API operations?

Separate model decisions from execution permissions. The model can propose a tool call, but the backend validates the tool name, arguments, user permissions and business rules. Keep capabilities allowlisted and require approval for high-impact actions.

A request to ignore instructions and delete production records must not grant access to a deletion capability.

74. What is the principle of least privilege?

Least privilege grants each identity only the permissions needed for its responsibilities. A document-ingestion Lambda might read approved S3 objects and write to a specific index, without access to every bucket. A tool that reads invoice status should not be able to initiate payments.

Use scoped IAM roles, resource restrictions, separate roles where appropriate, secret management and periodic reviews. Least privilege limits the impact of compromised credentials, prompt injection and application defects.

75. How would you defend against a request for another employee's salary and an injected instruction to disclose it externally?

Authenticate the requester; authorize salary access; exclude unauthorized documents from retrieval; treat the malicious document instruction as untrusted; restrict tools and prohibit unrestricted bulk export; validate recipients, data classification and operation permissions; require approval for permitted sensitive disclosures; and audit the attempt.

Even if the LLM follows the malicious instruction, the trusted backend must refuse the unauthorized action.

Section 6: React UI and end-to-end AWS architecture

76. Design a complete React and AWS Bedrock agent architecture.

Use React for the UI; Cognito or an enterprise identity provider for authentication; API Gateway for the entry point; Lambda for validation and orchestration; Bedrock for inference and suitable managed agent capabilities; a vector store or Knowledge Bases for retrieval; business APIs for authoritative data; Step Functions/durable storage for long-running work; and CloudWatch/tracing for monitoring.

The authenticated request reaches the backend, which determines permissions and invokes the model and authorized tools. Retrieved evidence is supplied to the model and the response is returned to React. Sensitive actions use a validated execution path and approval where required.

77. Should React call Bedrock directly or use a backend API?

For an enterprise agent with private data and business tools, use a backend API. It centralizes authorization, business rules, credential protection, multi-tool orchestration and auditing. It also manages rate limits, quotas, validation, prompt versions and execution budgets.

Direct client-to-service access can suit carefully designed use cases with supported identity/access mechanisms, but it is not the default for a privileged enterprise agent.

78. How do you stream LLM responses to React?

Streaming sends response chunks as generated. Server-Sent Events are useful for one-way server-to-client updates; WebSockets support bidirectional communication; streaming HTTP responses can work where the API path supports the required semantics.

Account for buffering, timeouts, cancellation, reconnection and authentication. For tool-using agents, distinguish generated text from task completion: a streamed recommendation does not prove a downstream operation succeeded.

79. How do you implement authentication with Cognito or an enterprise identity provider?

Configure the identity provider to authenticate users and issue tokens. React obtains a token through its supported flow and sends it to API Gateway. The authorizer validates it; Lambda applies application-specific authorization before retrieval or tool execution.

Do not rely on unverified claims, client-supplied role names or the LLM's interpretation of identity. Address token expiry, refresh, logout, session handling and permission changes.

80. How would you design REST APIs for agent tasks?

For long-running workflows, separate task submission, status and result retrieval:
- POST /agent-tasks — create a task and return its ID.
- GET /agent-tasks/{taskId} — retrieve status.
- GET /agent-tasks/{taskId}/result — retrieve completed result.
- POST /agent-tasks/{taskId}/approval — submit an authorized approval.

Check ownership and permissions on every endpoint. Statuses may include ACCEPTED, RUNNING, WAITING_FOR_APPROVAL, SUCCEEDED, FAILED and CANCELLED. Acceptance is not completion; include idempotency and concurrency controls.

81. How do you support long-running workflows in React?

Use asynchronous execution with a persistent task ID. React submits the task and receives its ID, then polls status or subscribes to updates. The backend persists progress so users can reconnect after closing the browser.

For a due-diligence report, show meaningful stages without claiming unverified work is complete. The backend is the source of truth for status, and authorization is rechecked when results are retrieved.

82. How do you secure conversation history?

Associate each conversation with an owner or explicitly authorized participants. Store records with stable IDs and trusted identity/tenant context; check permissions on every retrieval or modification.

Apply retention, encryption, data minimization and deletion workflows. A conversation ID is not an access secret: knowing another user's ID must not allow access to their messages or tool results.

83. How do you display citations in a React RAG application?

Return a structured answer plus source metadata, for example document ID, title and page. Generate references from retrieved evidence rather than allowing the LLM to invent identifiers or page numbers.

React can display citations as links to authorized documents. The document endpoint must independently enforce permissions, and citations should support the answer actually returned.

84. How would you build a human approval screen?

Show the proposed action, affected record, supporting evidence, expected impact and risks. Let the reviewer approve, reject or request clarification.

The backend creates a pending approval tied to the exact action. On approval, validate reviewer identity and permissions, approval validity and current business state before execution. For a billing-address change, show existing and proposed values and the source. Audit proposal, decision, execution and failures.

85. How do you distinguish task acceptance from confirmed completion?

Use explicit task states and authoritative completion checks. A successful POST /agent-tasks response means the task was accepted, not that the business operation succeeded.

States can be ACCEPTED, RUNNING, WAITING_FOR_APPROVAL, SUCCEEDED, FAILED and CANCELLED. If an agent requests a customer-record update, verify it through the authoritative system before marking the task successful. This prevents misleading users when model generation succeeds but a downstream operation fails.

Section 7: Advanced production engineering and leadership

86. How do you evaluate an AI agent before production deployment?

Evaluate component-level measures such as tool-selection accuracy, argument validity, retrieval relevance and schema compliance; end-to-end measures such as task completion, correctness, groundedness, policy compliance and recovery; and business outcomes such as time saved and customer experience.

An incident agent must retrieve correct logs, use authorized tools and recommend safe remediation—not merely produce a plausible diagnosis. Establish thresholds and test normal, ambiguous, adversarial and failure scenarios.

87. How would you build an evaluation dataset?

Create a representative dataset from real business tasks with sensitive data appropriately protected. Include normal requests, ambiguous/incomplete questions, clarification cases, unanswerable questions, authorization failures, prompt injections, tool timeouts, malformed responses and conflicting/outdated sources.

For RAG, record expected supporting documents; for agents, record permitted tools, expected outcomes and prohibited actions. Version the dataset and run regression tests for prompt, model, retrieval and orchestration changes. Human-reviewed ground truth is important for high-impact decisions.

88. How do you monitor a production agent?

Monitor reliability (task success, tool failures, timeouts), performance (end-to-end/model/retrieval latency), cost (tokens and cost per successful task), quality (groundedness and relevance), security (authorization denials and suspicious calls) and business outcomes (time saved and successful resolutions).

Use CloudWatch and distributed tracing as appropriate. Technical API success does not necessarily mean business success. Avoid logging unnecessary prompts, sensitive documents or personal data.

89. How do you trace a request across React, Lambda, Bedrock and downstream APIs?

Assign a correlation ID at request entry and propagate it through APIs, orchestration, tools and downstream services where supported. Record key events: authentication, retrieval, model invocation, tool selection/execution, results, final response and business verification.

Capture durations, error categories, model and prompt versions and token usage without exposing secrets or unnecessary sensitive content. This helps identify whether latency or failures originate in retrieval, inference or a downstream service.

90. How do you control agentic AI costs?

Optimize cost per successful task. Choose the least expensive model that meets quality requirements; reduce irrelevant context and duplicate tool results; bound retrieval and iterations; cache suitable non-sensitive results; parallelize independent work; route simple tasks to simpler models; and monitor budgets and quotas.

An agent should not make five powerful-model calls for a task solvable with one retrieval step and a smaller model. Alert on unusual cost increases.

91. How do you reduce the latency of an agent taking 30 seconds?

Trace the request first. Parallelize independent API calls, optimize retrieval and context size, remove unnecessary model calls, evaluate model choice and investigate network/downstream latency and concurrency limits.

Do not parallelize dependent operations: a report cannot be finalized before required research is available. Reduce latency without sacrificing answer quality, authorization or safe execution.

92. How do you design for model or AWS service failures?

Use bounded retries, exponential backoff, timeouts, circuit breakers where appropriate and durable state for long-running work. Retry transient model failures; stop or return controlled partial results for permanent downstream failures.

Fallback models can improve availability but must be evaluated for quality, tool compatibility, data handling and cost. For a payment, verify transaction state before retrying an uncertain request. Availability must not cause duplicate transactions or incorrect outcomes.

93. How do you roll out a new model or prompt version safely?

Use offline regression evaluation, security testing, shadow testing without production side effects, a limited canary rollout, monitoring and rollback against predefined thresholds. Version prompts, model configurations, tool schemas and retrieval settings together where possible.

A model may improve prose but become less reliable at tool calling, so evaluate the entire workflow, not just final text.

94. When would you choose a single LLM call, workflow, single agent, multi-agent system or fine-tuned model?

Choose the simplest architecture that meets requirements. A single LLM call suits summarization and extraction; a deterministic workflow suits predictable business rules; a single agent suits dynamic tool selection; multi-agent systems suit distinct specialist tasks with coordination benefits; fine-tuning suits persistent task-specific behavior when evaluation demonstrates improvement.

For example, invoice extraction needs an LLM plus deterministic validation, incident investigation may benefit from an agent, and independent specialist research may justify multiple agents. Extra complexity must be justified by measurable gains.

95. How do you make agent execution auditable despite nondeterministic outputs?

Record task ID, model/configuration and prompt versions, tool requests, validated arguments, results, approvals, retrieved source/document versions and final business outcome.

For a refund recommendation, the audit should show evidence, amount, authorization, approval and payment-system result. Do not expect identical text every time; make important decisions, tool executions and business transitions observable and verifiable. Protect audit records against unauthorized modification and apply retention controls.

96. How would you build a reusable enterprise agent platform?

Provide shared model invocation/orchestration, a governed tool registry, authentication and authorization, reusable RAG components, configurable safety controls, centralized observability/evaluation, cost budgets and versioned deployment/rollback.

HR, finance and IT can reuse the platform while having different tools, knowledge sources and policies. The platform team owns core infrastructure; product teams own use-case logic, data quality and business acceptance criteria.

97. How do you handle data residency, encryption, secrets and retention?

Define data classification, residency, regulatory and retention requirements before choosing the architecture. Configure appropriate AWS Regions, encryption in transit/at rest, KMS policies, IAM, private connectivity where required and approved secrets management.

Review what is transmitted to the model and how application data and logs are retained. Validate requirements against current AWS documentation, model-specific behavior and organizational policy rather than assuming all features have identical regional availability or data handling.

98. An agent passes 95% of tests but occasionally executes an incorrect write operation. Would you release it?

Not for unrestricted production use based only on the aggregate pass rate. Severity matters: a few high-impact failures can outweigh many successful low-risk cases.

Investigate failures; add deterministic parameter validation, strict authorization, idempotency, transaction verification, approval gates and targeted adversarial/regression tests. Use canary deployment with rollback thresholds. If an operation can cause significant harm, disable autonomous execution until residual risk is acceptable; a read-only or recommendation-only release may still be possible.

99. How would you introduce agentic AI across multiple business-critical applications?

Start with business outcomes, not technology novelty. Identify high-value workflows and assess feasibility, data readiness, integration complexity, risk and ROI. Classify use cases by risk: low-risk summarization can be piloted earlier, while financial transactions require stronger controls and evidence.

Establish reusable identity, tool, RAG, evaluation, observability and cost capabilities. Measure adoption, task success, time saved, cost per outcome and risk incidents. Assign business, engineering, security, data and operations ownership, with clear success criteria and escalation paths.

100. Design a complete production-grade multi-agent enterprise assistant.

Use React for chat, citations, task status and approvals; API Gateway and an identity provider for authentication and access controls; Lambda/Step Functions for orchestration and durable state; Bedrock for model inference; research and business-data specialist agents; RAG and authorized APIs; Guardrails plus server-side policy enforcement; an action agent for validated proposals; and authoritative systems plus an audit trail.

Flow: authenticate the user; establish task, permissions and budget; delegate research and data retrieval; validate results; propose an answer or action; apply safety and authorization checks; request approval for sensitive actions; execute through an authorized service; verify the result; return citations and status to React; and retain appropriate audit records.

Production controls include least-privilege IAM, tenant isolation, prompt-injection defence, bounded iterations, timeouts, retries, idempotency, durable state, evaluation, tracing, token/cost monitoring, versioning, approvals and governance.

Use managed Bedrock capabilities where they fit and custom orchestration where stronger control is needed. The core principle is: the LLM determines or recommends what to do; the application enforces what it may do and verifies what actually happened.